

Django 教學 08

DTL進階與元件整理

第 14 章

從最小範例到專案實作與驗收

本份學習路線

先完成 [07_資料列表搜尋與分頁](#)。

- **第 14 章：Template 工具與責任分工**

每章依序：概念、最小範例、語法、專案對照、實作與驗收。

[全課目錄](#) · [實作手冊對照](#)

第 14 章

Template 工具與責任分工

整理一個模板元件，保留查詢狀態，說明顯示邏輯與業務規則的分工。

本章的操作環境與成果

前台實作選 LearnBoard 或 LearnMart；圖解與最小範例先說明概念，再對照現有程式。

完成成果： 整理一個模板元件，保留查詢狀態，說明顯示邏輯與業務規則的分工。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。
進階頁可回查，但所有基本驗收需完成。

14-1 工具箱的範例資料與使用方式

本章的 article、order、tags、upload 是各頁示意的 context 變數。它們不代表 catalog 或 LearnBoard 已有這些欄位。

- 先用小型 dict／list 或現有資料建立需要的 context。
- 常用工具完成實作；低頻格式化、分組與自訂 tag 可在需要時回查。
- LearnJournal 補強頁為延伸閱讀，不列入本課先備。
- 第 4 章已學日期、金額、預設值及文字截斷，這裡直接應用。

14-2 DTL 的定位

Template 應該負責：

- 呈現資料
- 小型格式轉換
- 簡單條件與迴圈
- 組合 reusable partial

不應該負責：

- 複雜 business rule
- database query orchestration
- 大量 Python 計算
- permission 核心判斷

複雜邏輯移到 view / model / QuerySet / custom template tag 。

14-3 相對時間：`timesince`

```
{{ article.created_at|timesince }} 前
```

可能顯示：

3 天, 4 小時 前

適合：

- 留言時間
- 發文多久
- 最近活動

14-4 未來還有多久：`timeuntil`

距離截止還有 `{{ deadline|timeuntil }}`

適合：

- 活動倒數
- 排程發佈
- 訂單付款期限

如果要精準倒數到秒，應交給 JavaScript，而不是 template 每秒重算。

14-5 Template 裡取得現在時間：{% now %}

```
{% now "Y-m-d H:i" %}
```

也可以存進變數：

```
{% now "Y" as current_year %}  
© {{ current_year }}
```

適合 footer 等簡單呈現。

14-6 更自然的時間：`humanize`

先載入：

```
{% load humanize %}
```

常用：

```
{{ article.created_at|naturaltime }}  
{{ article.published_at|naturalday }}
```

可能得到：

```
3 minutes ago  
今天  
明天
```

實際語言依 `i18n` 設定。

14-7 大數字： `intcomma` / `intword`

```
{% load humanize %}  
{{ article.view_count|intcomma }}  
{{ total_users|intword }}
```

例：

```
12,345  
1.2 million
```

適合 dashboard / analytics 類畫面。

14-8 HTML 截斷版本要小心

Django 也有 HTML-aware variants，例如：

```
{{ trusted_html|truncatechars_html:120 }}
```

重點：

HTML-aware 不等於 XSS sanitizer。

如果內容來自使用者，仍要先有可信的 sanitization policy。

14-9 檔案大小：`filesizeformat`

```
{{ upload.size|filesizeformat }}
```

可能顯示：

2.4 MB

比直接顯示 `2516582` bytes 更友善。

LearnMart 圖片上傳教材可以直接使用。

14-10 length / wordcount

```
{{ articles|length }}  
{{ article.body|wordcount }}
```

適合純呈現。

如果 `articles` 是 `QuerySet`，而且你只是要 DB count，通常 `view/queryset` 的 `.count()` 更能表達意圖。

14-11 List helpers

```
{{ tags|first }}  
{{ tags|last }}  
{{ tags|join:", " }}  
{{ articles|slice:":3" }}
```

適合小型 presentation transformation 。

不要為了只顯示前三筆而在大量 QuerySet evaluate 後才 slice ；能在 ORM 限制就優先在 ORM 。

14-12 dictsort

```
{% for item in items|dictsort:"name" %}  
  {{ item.name }}  
{% endfor %}
```

可用於 template 收到的普通 dict/list data。

如果排序是 domain rule 或會影響 pagination，應在 ORM / view 排好。

14-13 **yesno**

```
{{ order.is_paid|yesno:"已付款,未付款" }}
```

也可給 None 第三種：

```
{{ value|yesno:"是,否,未知" }}
```

適合 Boolean label。

14-14 pluralize

英文 UI 常用：

```
{{ count }} comment{{ count|pluralize }}
```

中文通常不需要單複數變化，但看官方教材或國際化專案時很常遇到。

14-15 `urlize` / `urlizetrunc`

```
{{ plain_text|urlize }}  
{{ plain_text|urlizetrunc:30 }}
```

把純文字 URL/email 轉成 link。

注意：它不是 Markdown parser，也不是 sanitization 工具。

14-16 {% with %} : 替長 lookup 取名字

```
{% with total=order.items.count %}  
  共 {{ total }} 項  
{% endwith %}
```

適合：

- 提升可讀性
- 同一個值在區塊內重複使用

不要拿 {% with %} 取代應該在 view 計算的複雜資料。

14-17 {% firstof %} : 多個 fallback

```
{% firstof user.get_full_name user.username "匿名" %}
```

相當於「第一個 truthy 值」。

比多層 {% if %} 簡潔。

14-18 {% cycle %} : 輪流值

```
{% for row in rows %}  
<tr class="{% cycle 'odd' 'even' %}">
```

常用：

- zebra table
- card variant
- alternating alignment

Bootstrap 已能處理很多 styling，但讀舊模板時很常見。

14-19 {% ifchanged %} : 分組顯示

```
{% for article in articles %}
  {% ifchanged article.category.name %}
    <h2>{{ article.category.name }}</h2>
  {% endifchanged %}
  ...
{% endfor %}
```

資料必須先按 grouping key 排好。

14-20 {% regroup %} : Template 層分組

```
{% regroup articles by category as grouped %}
```

```
{% for group in grouped %}  
  <h2>{{ group.grouper }}</h2>  
  {% for article in group.list %}  
    {{ article.title }}  
  {% endfor %}  
{% endfor %}
```

適合「已經排序好的資料」做 presentation grouping。

不是 SQL `GROUP BY` 的替代品。

14-21 {% querystring %} : 分頁超實用

Django 5.1+ 內建：

```
<a href="{% querystring page=page_obj.next_page_number %}">  
    下一頁  
</a>
```

如果目前 URL 是：

```
/?q=django&category=web&page=1
```

它能只改 `page`，保留其他 query parameters。

14-22 搜尋＋分頁的典型寫法

```
{% if page_obj.has_previous %}
<a href="{% querystring page=page_obj.previous_page_number %}">
    上一頁
</a>
{% endif %}

{% if page_obj.has_next %}
<a href="{% querystring page=page_obj.next_page_number %}">
    下一頁
</a>
{% endif %}
```

手動拼接 `?q={{ query }}&page=...` 只保留 q，容易漏掉其他篩選條件。

14-23 json_script : 安全把資料交給 JS

```
{{ chart_data|json_script:"chart-data" }}
```

JavaScript :

```
const data = JSON.parse(  
  document.getElementById("chart-data").textContent  
);
```

比直接把 Python data 插進 `<script>` 字串安全，也更符合 CSP。

14-24 Autoescape 是預設安全線

```
{{ comment.body }}
```

Django 預設會 escape HTML 特殊字元。

例如 user 輸入：

```
<script>alert(1)</script>
```

不應直接成為可執行 script。

14-25 **safe** 要非常克制

```
{{ value|safe }}
```

它的意思不是「幫我清理 HTML」。

它的意思是：

我向 template engine 保證，這段內容已可信，可不要 escape。

如果保證錯了，就是 XSS。

14-26 `striptags|safe` 不是 sanitizer

不要假設：

```
{{ user_html|striptags|safe }}
```

就一定安全。

Django 官方文件也明確提醒 `striptags` 的輸出不應再直接標記 `safe`。

有 user HTML 需求時，用專門 sanitizer policy。

14-27 Template 裡不要呼叫帶參數方法

DTL 允許：

```
{{ article.get_absolute_url }}
```

但不支援像 Python：

```
article.some_method("x")
```

需要帶參數的 presentation logic：

- 先在 view 計算
- 自訂 filter
- simple_tag / inclusion_tag

14-28 什麼時候做 custom filter ?

當你一直重複：

```
{{ value|...一串處理... }}
```

而且它是純 presentation transformation 。

例：LearnJournal 的 `markdownify` 。

Custom filter 應盡量是 deterministic、沒有 DB side effect 。

14-29 什麼時候做 inclusion tag ?

當一小塊 UI 同時需要：

- 自己的 query/context
- 自己的 partial template
- 多頁重用

例：

```
{% latest_articles 5 %}  
{% tag_cloud 20 %}
```

比每個 view 都塞相同 context 更乾淨。

14-30 LearnBoard 可以立刻用的補強

留言清單：

```
{{ message.content|truncatechars:120 }}  
{{ message.created_at|naturaltime }}
```

搜尋分頁：

```
{% querystring page=page_obj.next_page_number %}
```

14-31 LearnMart 可以立刻用的補強

```
{{ product.description|truncatewords:30 }}  
{{ product.image.size|filesizeformat }}  
{{ product.rating|floatformat:1 }}  
{{ order.created_at|date:"Y-m-d H:i" }}
```

數量：

```
{{ cart_items|length }} 項商品
```

14-32 LearnJournal 可以立刻用的補強

```
{% load humanize %}
{{ article.published_at|naturaltime }}
{{ article.view_count|intcomma }} 次瀏覽
{{ article.excerpt|truncatechars:140 }}
```

Scheduled :

```
距離發佈還有 {{ article.published_at|timeuntil }}
```

14-33 判斷「該放哪一層」

如果你正在 template 想做：

- 多次 DB query
- 複雜排序
- permission decision
- transaction
- API call
- 多層資料轉換

停下來。

大多數情況應該移到 Python 層。

14-34 本章實作題

1. LearnBoard 留言列表加 `naturaltime` 。
2. LearnMart 商品卡改用 `truncatechars` / `floatformat` 。
3. 任一搜尋分頁改用 `{% querystring %}` 保留 `q` 。
4. 做一頁展示 `default` 和 `default_if_none` 對 `0` 的差別。
5. 用 `json_script` 把 5 個文章瀏覽數交給前端 console 印出。

14-35 參考資料

- Django 6.1.1 Template language :
<https://docs.djangoproject.com/en/6.1/ref/templates/language/>
- Built-in tags / filters :
<https://docs.djangoproject.com/en/6.1/ref/templates/builtins/>
- Humanize : <https://docs.djangoproject.com/en/6.1/ref/contrib/humanize/>
- Time zones in templates :
<https://docs.djangoproject.com/en/6.1/topics/i18n/timezones/>

14-36 最後記住三件事

1. 格式化留在 `template`，`business rule` 留在 `Python`。
2. `autoescape` 是預設安全線，不要隨便 `safe`。
3. 如果同一段 `presentation logic` 重複出現，就考慮 `custom filter/tag`。

第 14 章實作與離堂檢核

任務： 整理一個模板元件，保留查詢狀態，說明顯示邏輯與業務規則的分工。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

配套練習： 完成本章工具箱實作題，保留模板前後差異與畫面。

本份完成與後續

下一份：[09_表單與資料驗證](#)。

- 保留本份操作紀錄，確認使用正確的專案與資料庫。
- 章節與實作對應可由 [全課目錄](#) 回查。